Organizing Structures

Ruby has two basic concepts for organizing methods: classes and modules. We cover each in turn.

Classes

Here's a Ruby class definition:

```
1:  class Order < ApplicationRecord
 -    has_many :line_items
 -    def self.find_all_unpaid
 -      self.where('paid = 0')
5:    end
 -    def total
 -      sum = 0
 -      line_items.each {|li| sum += li.total}
 -      sum
10:   end
 -  end
```

Class definitions start with the `class` keyword and are followed by the class name (which must start with an uppercase letter). This `Order` class is defined to be a subclass of the `ApplicationRecord` class.

Rails makes heavy use of class-level declarations. Here, `has_many` is a method that's defined by Active Record. It's called as the `Order` class is being defined. Normally these kinds of methods make assertions about the class, so in this book we call them *declarations*.

Within a class body, you can define class methods and instance methods. Prefixing a method name with `self.` (as we do on line 3) makes it a class method; it can be called on the class generally. In this case, we can make the following call anywhere in our application:

```
to_collect = Order.find_all_unpaid
```

Objects of a class hold their state in *instance variables*. These variables, whose